

Adora — Handoff for PINN Spectropolarimetric Inversion

Code summary & extension plan · Ivan Milic · 2026-09-18 · 7

The idea in one line. Adora’s entire forward model — populations → opacities → polarised formal solution — is *one differentiable JAX function (fp64)*, so **response functions and Jacobians come for free**. A PINN inversion is simply: replace the fixed *nodes* of `nodes.py` with a coordinate MLP $f_\theta(\gamma(\mathbf{x}))$ that emits the 8 atmospheric parameters, push it through the *unchanged* forward model, and train θ with `optax`. Gradients already flow Stokes → RT → atmosphere → θ .

The reusable core (copy these)

- `lineop.py` — *heart*: `read_kurucz`→`AtomicData` pytree (precomputes LS-coupling Zeeman); LTE line η, χ ; Fe I/II/III pops (Irwin 1981); Planck source.
- `contop.py` — continuum (H bf, H⁻ bf/ff Gray, Saha). `voigt.py` — Voigt–Faraday (Humlicek w4).
- `vector_formal_solver.py` — polarised **DELO** (Janett 2017), 4×4 matrix.
- `vector_response_fn.py` — `lte_polarised_rt`: the one call a PINN makes (validated vs. Lightweaver, Fig.).
- Scalar-only twin: `response_fn.py` + `scalar_formal_solver.py`. Data: `kurucz.6301.6302.linelist`.

The single forward call (per λ , per column)

```
I = lte_polarised_rt(adata, wave, dz, T, ne,
                    nhtot, vz, vturb, b, gamma_b, chi_b)
returns [I, Q, U, V]; vmap over wave → profile. The 8
per-depth parameters (what the network outputs) are
T, ne, nH, vz, vturb, B,  $\gamma_B$ ,  $\chi_B$ ; dz is the layer spacing.
Bounds/output transforms: reuse nodes.py (:269--288).
```

Current inversion = the baseline the PINN replaces

`nodes.py` (NodeSpec) maps a flat node vector → `interpax` interpolation to a depth grid → the atmosphere tuple, fit by SciPy `trf` with a JIT `jacrev` Jacobian (`iterate_scipy.py`); `iterate_adam.py` is the `optax` loop — **the template for the PINN trainer**. `reconstruct_from_nodes` is precisely the function a neural field supersedes.

PINN plan (extends `adora.3_plan_neuralfield`)

Encoding $\gamma(\mathbf{x}) = [\sin, \cos](2\pi\mathbf{B}\mathbf{x})$, per-axis bandwidth (z finer). **Net** 3–5 layers, width 64–256 → 8 outputs, each through a physical map (softplus/exp for densities & v_{turb} , sigmoid for T , tanh for v_z , bounded B, γ_B, χ_B). **Loss** χ^2 misfit + regularisers (depth smoothness, hydrostatic/EOS residual, opt. $\nabla \cdot \mathbf{B} = 0$). **Train** Adam→L-BFGS, `jit+vmap` over pixels/ λ . **Prove** single pixel → 2D map (shared θ = spatial coherence, the payoff) → invert a synthetic FALC/MURaM cube vs. the node baseline. **New files:** `neural_field.py`, `invert_field.py`.

Conventions & gotchas that must travel

- **fp64 mandatory** (`jax_enable_x64` before any `jnp`); **units** I in $\text{kW m}^{-2} \text{nm}^{-1} \text{sr}^{-1}$, λ in nm, B in Tesla, rest SI.
- **Depth order:** index 0 = *deepest* layer; $I_{\text{start}} = \text{Planck}(T_0)$ deep boundary; DELO integrates upward. The sampler must honour this.
- $\mu = 1$ **only** (no LOS projection yet); **no self-consistent EOS** — n_e, n_H are *free* (degenerate; the physics regularisers exist to constrain them).
- **lightweaver is a build-time dep** of `read_kurucz/nodes.py` (not pip-installable here). For portability: call `read_kurucz` once and *serialise the AtomicData pytree* (plain arrays) so the runtime path drops the dependency.
- **Fe only** (Irwin partition fns); line data validated only for the 630 nm pair.

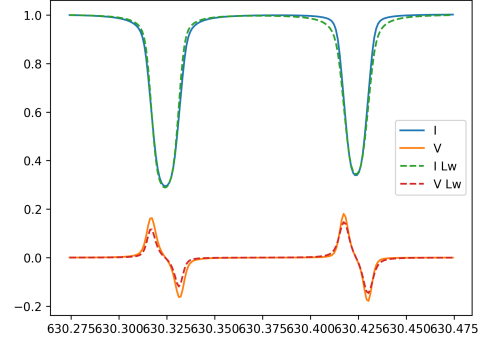


Figure 1: Stokes I, V for the 630 nm Fe I pair from Adora’s polarised DELO solver (solid) vs. Lightweaver (dashed). The forward model is validated *and* end-to-end differentiable — the property the PINN relies on.

Full detail (call graph, fileline map, copy list, open questions) in `3dtesting/PINN_HANDOFF.md`. No code was run this session (reading only); Fig. 1 is from a prior validated run.